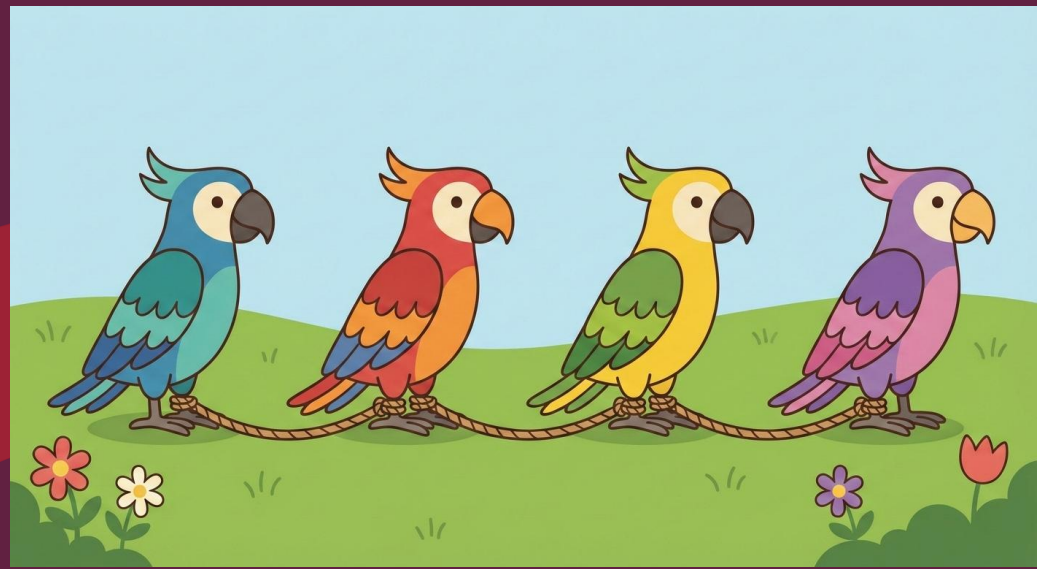
Agentic AI with LangChain and LangGraph

Presentation 5: Some Advanced Topics of LangChain

Primary Instructor: Michael Z
RA: YH

Date: 2026.Jan-Feb



Sobey School of Business
Saint Mary's University
MBAN

Saint Mary's University

*Quick Notes:

I would skip some topics for some reasons: Long-Term memory (current technique is not mature) and Multi-agent (Because it belongs to our next topic, LangGraph) and Context Engineering (It looks like philosophy of being AI engineer, rather than a topic of technique).

I would place different weights on different sections.

I mean, it could take me a 3-credit course to go over RAG (Retrieval Augmented Generation), but I only have couple mins to talk about it here. Thank you for your understanding in advance.

LangChain Inc is maintaining v0.3+ and v1.0+.

0. RAG

0. RAG: Core Idea

Sobey School of Business
Saint Mary's University
MBAN

The core idea of RAG can't be easier:

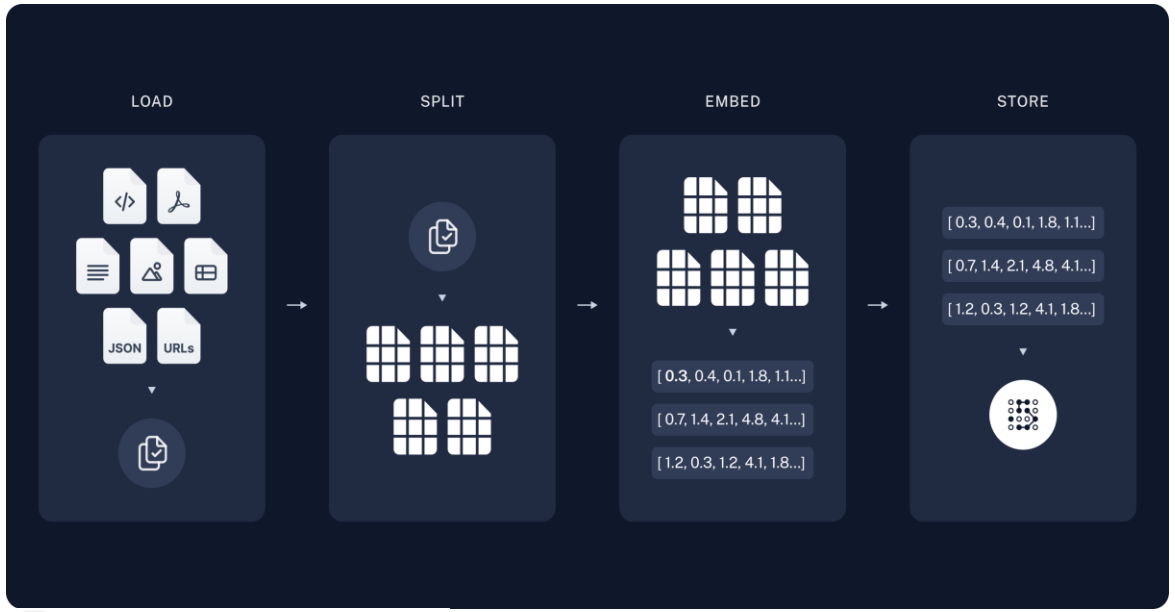
1. Documents Loading (Serialization)
2. Documents Splitting (also known as Chunking Strategy in some framework)
3. (Chunks) Embedding
4. Storing
5. Semantic Search for given query
6. Top K Records retrieved
7. Reranking (Optional)
8. Sent to LLM context for augmented generation

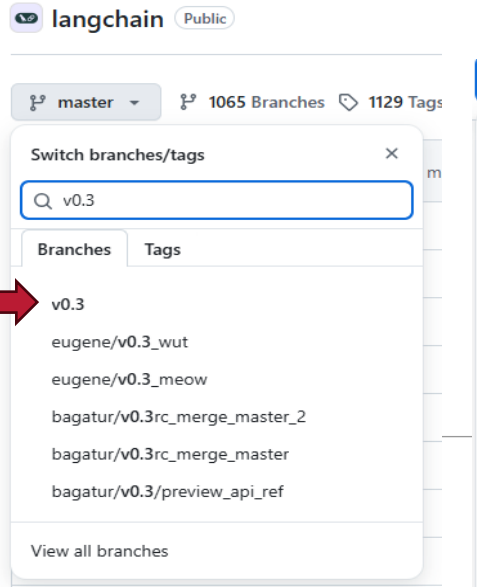
The image RHS covers 1-4.

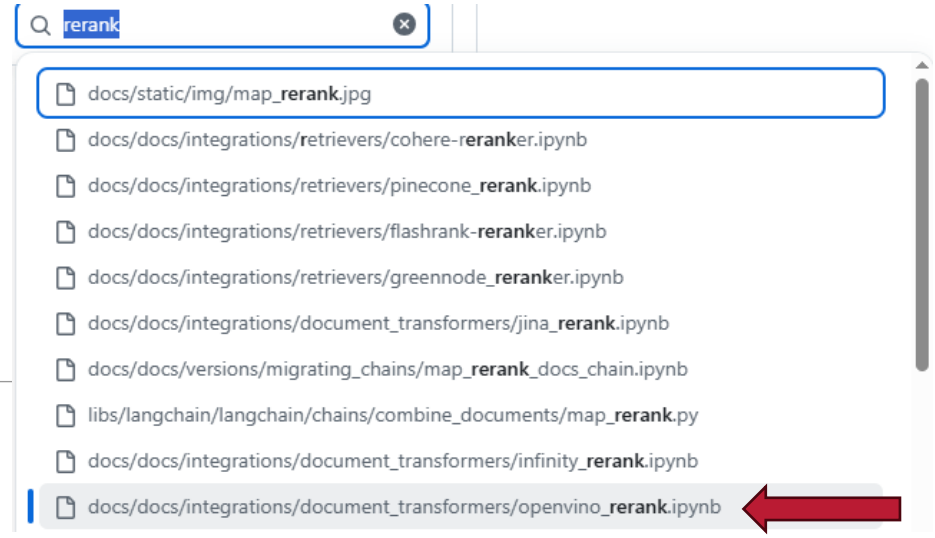
1-7 is R and 8 is G.

If you are interested in any of the steps mentioned above, raise your hands to ask me right now or google it on your own.

Current docs provided by LangChain didn't explain Reranking very well so you might go to its v0.3 to read archived docs in GitHub.



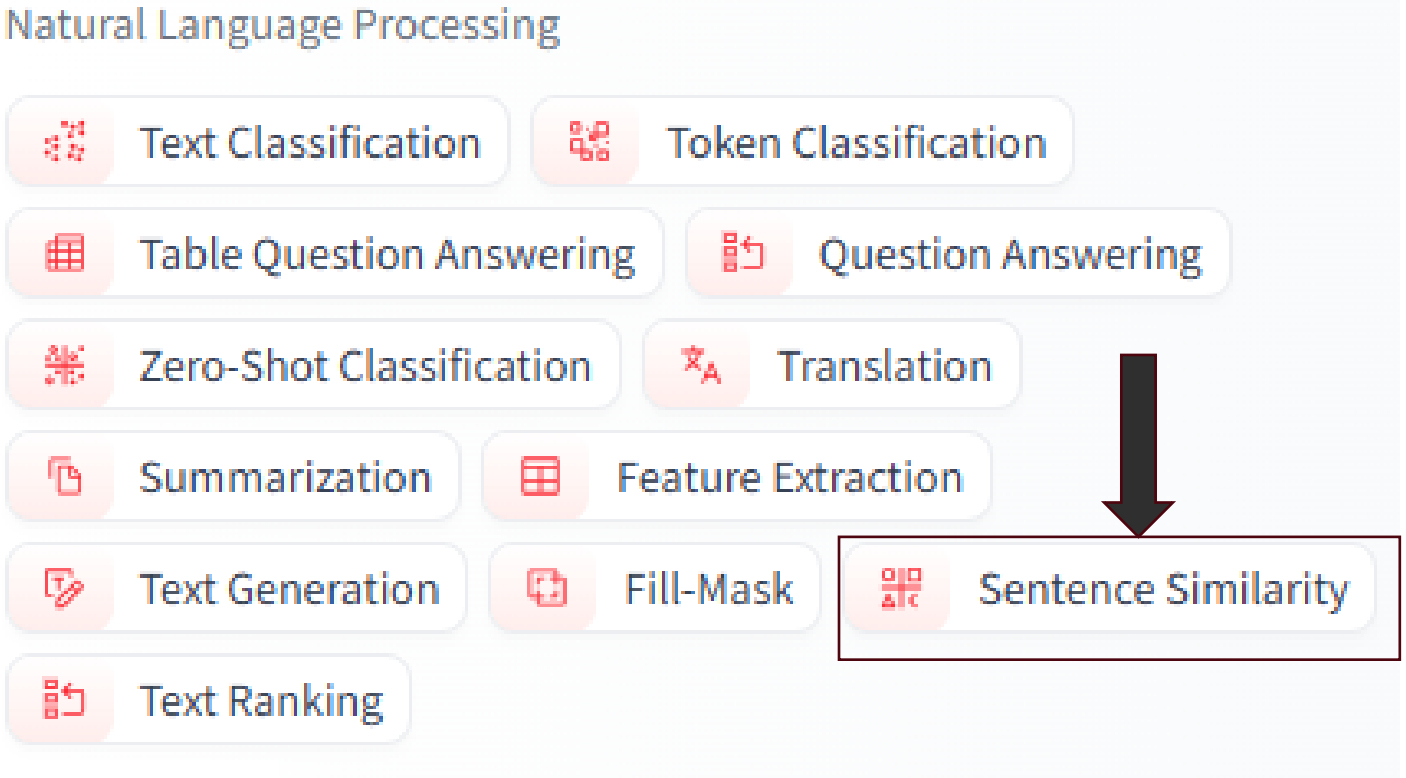




*Quick Note Again……

When you are playing with RAG and try to choose the embedding model you like from HuggingFace, you might notice that there doesn't actually exist a section called "Embedding Model".

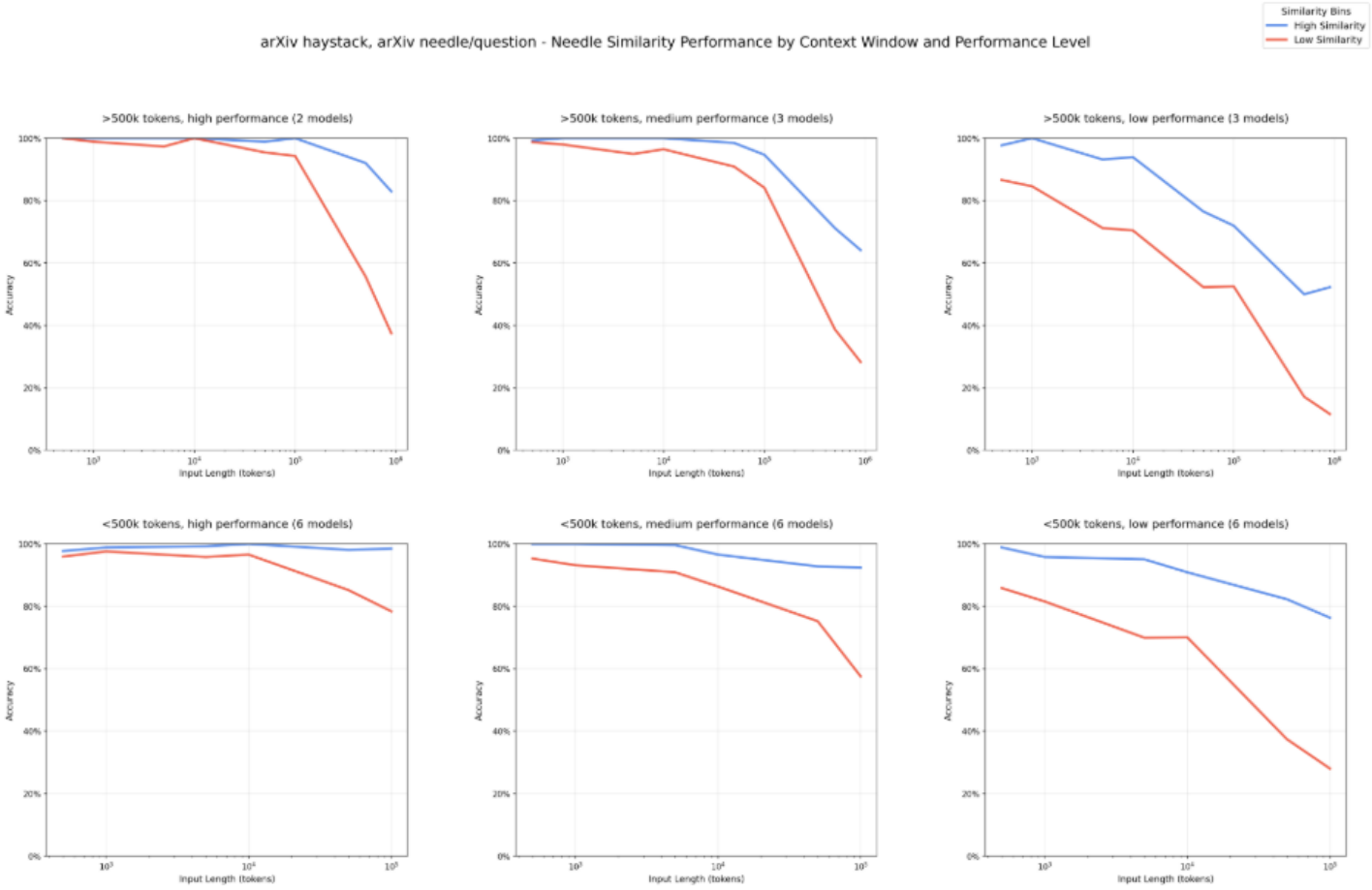
It's actually called "sentence similarity", which is a little bit weird but also comprehensible.



0. RAG: RAG is dead……But you still need to learn!

Results

We observe a clear pattern that performance degrades more quickly in input length with lower similarity needle-question pairs.



CHROMA TECHNICAL REPORT

July 14, 2025

Context Rot: How Increasing Input Tokens Impacts LLM Performance

Source: https://research.trychroma.com/context-rot

0. RAG: RAG in LangChain Docs

All, I mean all, variants of RAG you can see right now or in the future implement the core idea I discussed in previous slides, with

a little

a little

a little

a little

a little

a little

a little

a little

bit of engineering design added.

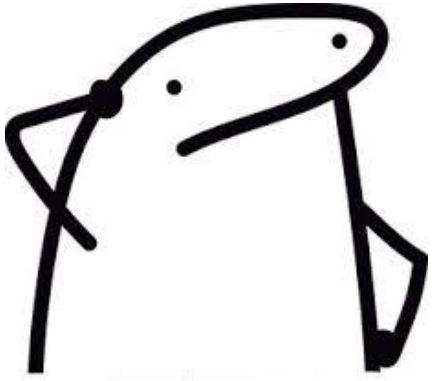
1. Vanilla RAG
2. Hybrid RAG
3. Rerank-based RAG
4. Query Expansion RAG
5. Multi-Hop RAG
6. Graph RAG
7. Adaptive RAG
8. Agentic RAG

……

*Fun Fact

When reading the LangChain docs (Hybrid RAG), I noticed the following, which reminds me of something.

- **Query enhancement**: Modify the input question to improve retrieval quality. This can involve rewriting unclear queries, generating multiple variations, or expanding queries with additional context.



Aha!!!

It turns out that it's actually MultiQueryRetriever (in v0.3 of LangChain)

How to use the MultiQueryRetriever

Distance-based vector database retrieval embeds (represents) queries in high-dimensional space and finds similar embedded documents based on a distance metric. But, retrieval may produce different results with subtle changes in query wording, or if the embeddings do not capture the semantics of the data well. Prompt engineering / tuning is sometimes done to manually address these problems, but can be tedious.

The MultiQueryRetriever automates the process of prompt tuning by using an LLM to generate multiple queries from different perspectives for a given user input query. For each query, it retrieves a set of relevant documents and takes the unique union across all queries to get a larger set of potentially relevant documents. By generating multiple perspectives on the same question, the `MultiQueryRetriever` can mitigate some of the limitations of the distance-based retrieval and get a richer set of results.

Let's build a vectorstore using the LLM Powered Autonomous Agents blog post by Lilian Weng from the RAG tutorial:

1. Human-In-The-Loop (HITL)

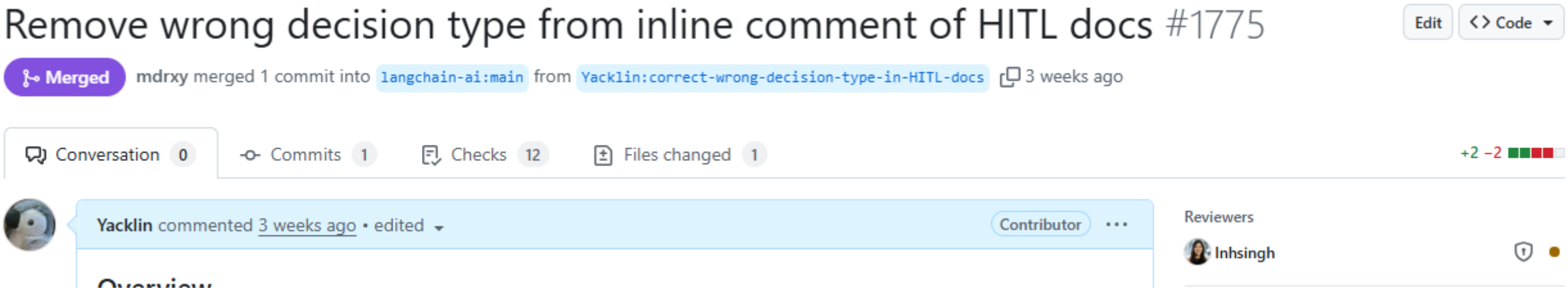
1. HITL

HITL is commonly used in multi-agent system, but I think it's also good to know that it could be used in single agent application.

HITL basically means that you impose some restrictions on some tools and agent needs your explicit approval when it's about to call these tools.

(If you don't understand "allowed decisions" in their documents, replace "allowed" with "available" simply.)

I would go over HITL docs for you right now.
To make sure you are learning correct documents, I corrected one wrong inline comment for you.



1. HITL: If you are good at using HITL…..

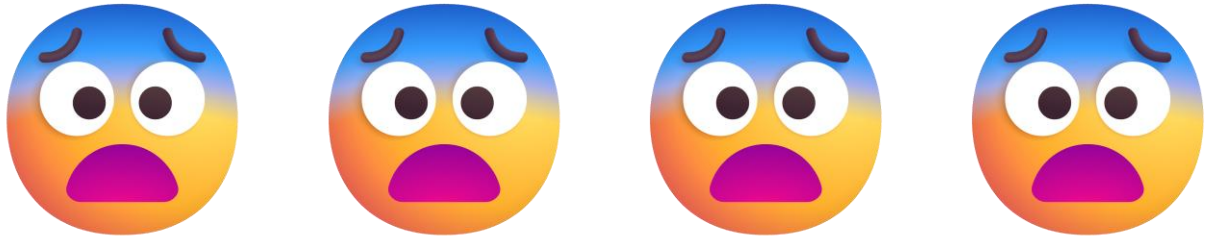
Those who are proud of being part of something or good at using something or loving something would call themselves "…..er".

For example, those who are good at singing would call themselves "singer".

If you are good at playing games, you might call yourself "gamer".

If you are good at using HITL..

HITL + ER = ….



2. MCP

*async and await keywords in Python

https://docs.python.org/3/library/asyncio-task.html

2. MCP: They finally accepted my fix!!!!!

I noticed some syntax errors in their docs when preparing slides last week, so I made a PR to fix it. But they didn't accept my fix until Thursday (Feb 5th). I believe the code snippet is quite copy-and-paste-able.

Fix syntax error for mcp docs #2386

Merged mdrxy merged 6 commits into `langchain-ai:main` from `Yacklin:fix-syntax-error-for-mcp-docs` yesterday

Conversation 1 | Commits 6 | Checks 14 | Files changed 1

Yacklin commented last week — Contributor

Overview

2. MCP

You make the tools in the Python script in which agent is about to be invoked, before we talk about MCP today.

If there is a lot of tools and tool's implementation is complex, you might want to move these tools to somewhere else. You also would like to allow others to use the tools you made.
Then, MCP is your best choice. You put tools in a single script and deploy this script as a http server (aka MCP server and its default port is 8000).

To create a custom MCP server, use the **FastMCP** library:

pip uv

```pip install fastmcp```

To test your agent with MCP tool servers, use the following examples:

Math server (stdio transport) | Weather server (streamable HTTP transport)

```
from fastmcp import FastMCP

mcp = FastMCP("Weather")

@mcp.tool()
async def get_weather(location: str) -> str:
    """Get weather for location."""
    return "It's always sunny in New York"

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

At client side, just simply call these tools gracefully.

```
client = MultiServerMCPClient(
    {
        "math": {
            "transport": "stdio",  # Local subprocess communication
            "command": "python",
            # Absolute path to your math_server.py file
            "args": ["/path/to/math_server.py"],
        },
        "weather": {
            "transport": "http",  # HTTP-based remote server
            # Ensure you start your weather server on port 8000
            "url": "http://localhost:8000/mcp",
        }
    }
)

tools = await client.get_tools()
agent = create_agent(
    "claude-sonnet-4-5-20250929",
    tools
)
math_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what's (3 + 5) x 12?"}]}
)
weather_response = await agent.ainvoke(
    {"messages": [{"role": "user", "content": "what is the weather in nyc?"}]
)
```
```

*A Quick In-Presentation Practice Here

3. Runtime (Context)

3. Runtime (Context)

There are two parts in Runtime: Context and Store (skipped)

Context can be treated as user structured input, to help you understand it better.

4. Guardrails

4. Guardrails: PII

There are two ways to achieve guardrails: One is HITP and another is PII detection.
What guardrails can do is screening, like what could not be visible to LLM and what could be visible to user. I mean because of middleware, you can basically control visibility of every single message, literally.
We already discussed HITP so I would only talk about PII here.

PII means personally identifiable information.

Built-in PII types:

- `email` - Email addresses
- `credit_card` - Credit card numbers (Luhn validated)
- `ip` - IP addresses
- `mac_address` - MAC addresses
- `url` - URLs

Here is a very quick in-class test and I want you to finish it:
LangChain-provided example is not that good. Can you refine it for them? How? (hint: to make LLM generate something you need…..)

```
# Use a model to evaluate safety
safety_prompt = f"""Evaluate if this response is safe and appropriate.
Respond with only 'SAFE' or 'UNSAFE'.

Response: {last_message.content}"""

result = safety_model.invoke([{"role": "user", "content": safety_prompt}])

if "UNSAFE" in result.content:
    last_message.content = "I cannot provide that response. Please rephrase y

return None
```

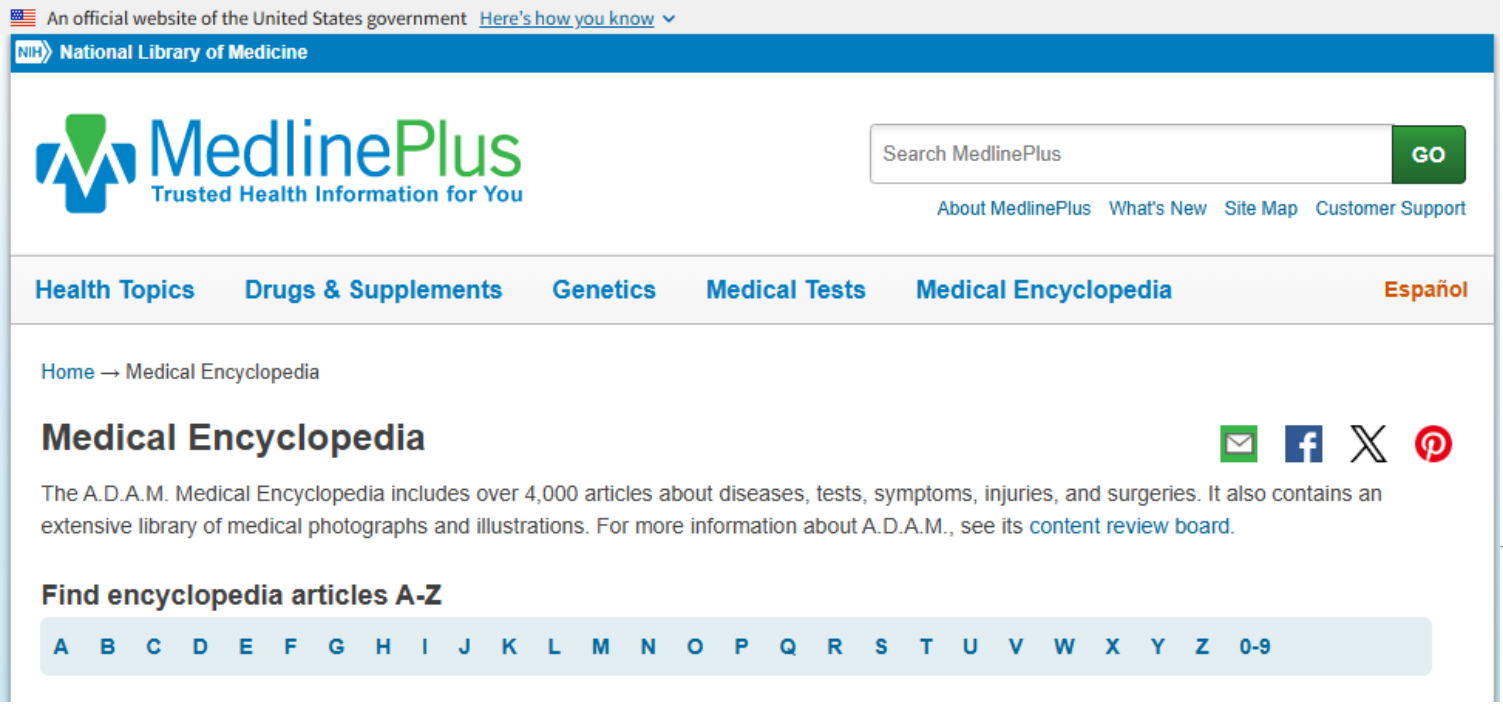
*I would take a couple questions here

5. Hands-On Project…..Again….

5. Hands-On Project: Agentic RAG vs Vanilla RAG

I believe most of you have used/built RAG-enabled applications before. Probably it's low-code or no-code. But this time I am curious if well-designed Agentic RAG could outperform Vanilla RAG.
Project Description: Build RAG AND Agentic RAG (which means two PROGRAMS) using the following website as external source of information and then compare quality of responses generated by these two approaches.
When user asks a healthcare related question, your Python programs, I mean both, should be able to retrieve relevant information from the website first, send it to Agent (or LLM) and then the Agent (or LLM) would respond to user with augmented response.

PLEASE DON'T SIMPLY SEND ALL OF WHAT IS RETRIEVED TO LLM/AGENT! USING TOKENS INEFFICIENTLY WOULD ONLY MAKE YOU RUN OUT OF 2-USD BUDGET!!!!!!!!!! ADD SOME FILTERS PLEASE!!!!!!




```python
@tool
def fetch_url(url: str) -> str:
    """Fetch text content from a URL"""
    response = requests.get(url, timeout=10.0)
    response.raise_for_status()
    return response.text
```

Sobey School of Business
Saint Mary's University
MBAN

5. Hands-On Project: A possibly helpful starter

Use bs4 to extract html content given a link.
Just simply: uv add bs4

uv add lxml to use lxml as feature.

Different page might or might not have the same html structure, so please click “inspect” to view the structure in the webpage.

I wont’ be able to teach bs4.

```python
import requests
from bs4 import BeautifulSoup

response = requests.get( url: "https://medlineplus.gov/ency/encyclopedia_U.htm", timeout = 10.0)
response.raise_for_status()
full_page_soup = BeautifulSoup(response.content,  features: "lxml")
blocks = full_page_soup.find("ul", id = "index").find_all("li")

for block in blocks:
    print(block.find("a").get("href"))
    print(block.find("a").get_text())
```

bs4_test

```
C:\Users\[redacted]\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\s5615543\PycharmProjects\PythonProject\bs4_test.py
article/000250.htm
Ulcerative colitis
patientinstructions/000936.htm
Ulcerative Colitis - children - discharge
patientinstructions/000203.htm
Ulcerative colitis - discharge
article/003228.htm
Ulcers
article/000789.htm
Ulnar nerve dysfunction
article/003336.htm
Ultrasound
article/003778.htm
Ultrasound pregnancy
```


Sobey School of Business Saint Mary's University | MBAN

5. Hands-On Project: How to Compare? (Evaluation)

If user asks a question like “what should I do if I have bipolar disorder and insomnia?”, a good RAG should retrieve both information related to bipolar disorder and information related to insomnia.

In other words, the more relevant information gets retrieved, the better the RAG is. This is how we, and most of researchers, evaluate RAG.

To make my life easier, please use print function to print what’s retrieved from website for each RAG.

Thank you for your paged attention!

arXiv > cs > arXiv:2309.06180

Computer Science > Machine Learning

[Submitted on 12 Sep 2023]

Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica